

Loophole, explained simply

2026-09-19 · @Someone

What we are building

We are building a tool that tries to break your own security code before a real attacker does. You give it a small piece of code that is meant to keep bad input out. The tool attacks that code again and again until it finds an input that sneaks through. Then it shows you the exact input that worked and the one line that closes the hole.

Think of it like a bouncer

Imagine you wrote a bouncer for the door of your app. The bouncer has one job: keep troublemakers out, let normal guests in.

Here is the problem. You only ever tested your bouncer by sending in polite, normal guests. Of course they were handled fine. You never once tried a troublemaker in a clever disguise.

Our tool is a professional troublemaker. It keeps trying disguises until one of them fools your bouncer, then it tells you exactly which disguise worked, so you can teach the bouncer to catch it next time.

A real example you can picture

Say you have a check meant to stop anyone from reaching files outside a safe folder. It watches for two dots in a row, because that is how people climb up folders, and it rejects any input that contains them.

Looks solid. But the same two dots can be written in a disguised form, `%2e%2e`, which means the exact same thing to the computer but does not match the simple check. It slips straight through.

A person might never think of that disguise. Our tool tries it in seconds, watches it walk past the check, and reports back: this input broke your guard, and here is the one line that fixes it.

Why it needs the newest AI

A simple tool would just throw random junk at your code and hope something breaks. It might try millions of random inputs and still never land on a real disguise like %2e%2e, because random noise almost never spells out an actual attack.

The newest AI is different. It already understands the tricks attackers use. So instead of guessing blindly, it reaches for the right disguise almost at once.

That is the whole reason this belongs on the newest model. A weaker model, asked whether some code is safe, tends to say it looks fine. The new one actually finds the way through.

Why we build an app, not just a chat

A fair question: if the AI is this smart, why not just paste your code into Claude and ask it to find the hole?

Because a chat gives you one answer and stops. Our tool does something a chat cannot. It actually runs your code, hundreds of times, watching what happens on each try and learning from it.

Think of it as a machine that keeps knocking on every door, notices which ones rattle, and focuses there. You cannot do that by hand in a chat window. The loop that runs and learns is the real product. The AI is just the clever part inside it.

Why this can win

The hackathon scores every project on the same four things. This idea hits all of them with one simple demo.

- It shows something only the newest model can do. The old model says it looks fine, the new one finds the break. That is exactly what the Breakthrough track asks for.
- The room sees it work with their own eyes. The code breaks live on the bad input, then holds after the fix. Nobody has to take our word for it.
- Everyone in that room has this exact problem. We all ship security code we never really tested. People will want this the moment they see it.

What we do first, so we do not waste the night

Before we build the whole thing, we run one quick test. We take three small security checks that we know have holes, and we race our smart version against the normal tools everyone already uses.

If our version finds the holes much faster, and never raises a false alarm on safe input, we have something special and we build it.

If it does not clearly win, we find that out in the first hour with a tiny script, not at 2am with half an app built. Then we drop it and pick another idea. No wasted night, nothing lost.

What we need to decide together

1. The name. Loophole is a working title. Mwanya, Swahili for a gap or opening, means the same thing and may land better with local judges. Your call.
2. The three test cases. We agree on three small security checks to attack. A folder path check, a database input cleaner, and an allowed list check are good candidates.
3. Who does what. One person owns the attacking loop, one owns the part that decides whether a break is real, one owns the on screen demo.
4. The stop rule. If the first quick test does not clearly win, we agree now that we stop and pick again, no arguments later.

A few words you will hear, in plain English

- Guard function. A small piece of code whose job is to block bad input. The bouncer.
- Bypass. An input that gets past the guard when it should have been blocked. The troublemaker who got in.
- Fuzzing. Throwing lots of inputs at code to see what breaks it. What our tool does, except smart instead of random.
- The loop. The part of our tool that keeps trying, running, and learning. The engine of the whole thing.